



COMPUTING PIPE DREAMS AND HEAT STREAMS



Emmon Das
EML3041 Spring 2025

Contents

Introduction	2
Part One: Fluid Velocity Profiling and Numerical Analysis of Laminar Pipe Flow	2
Chapter 1: Extracting data, converting it into base-10 and finding error	2
Questions For Chapter 1:.....	5
Chapter 2: Numerical Differentiation.....	6
Question For Chapter 2:	8
Chapter 3: Root finding	8
Questions for Chapter 3:.....	10
Part two: Computational Heat Transfer Analysis Through Composite Materials	11
Chapter 4: Linear System of Equations	11
Questions for chapter 4:	13
Chapter 5: Interpolation and Splines.....	14
Interpolation	15
Questions for Chapter 5 Interpolation:.....	16
Splines:	16
Questions for Chapter 5 Splines:	19
Chapter 6: Regression.....	19
Question for Chapter 6:	21
Part Three: Introduction to Variable Thermal Conductivity Integration	22
Chapter 7: Numerical Integration Using Trapezoidal Rule.....	22
Conclusion:	24

Introduction

In modern engineering, accurate simulation and analysis of physical systems are crucial for designing efficient and reliable components. This project, Computing Pipe Dreams and Heat Streams, bridges computational methods with core concepts from fluid mechanics and heat transfer. Through a structured, chapter-wise approach, we explore how numerical techniques such as binary decoding, numerical differentiation, root-finding, system solving, interpolation, regression, and numerical integration can be applied to solve real-world thermal and flow problems.

The project begins by examining laminar fluid flow in a cylindrical pipe, using the Hagen-Poiseuille equation to predict theoretical velocity profiles and comparing these with experimental data encoded in fixed-point binary format. We then transition to heat conduction across a multi-material wall, where temperature distributions are determined through linear system solving and further refined using interpolation and spline methods. Finally, we investigate regression modelling and integrate variable thermal conductivity to evaluate total heat transfer through non-uniform materials. Each section not only reinforces physical intuition but also demonstrates the power of MATLAB and numerical algorithms in translating equations into tangible engineering insights.

Part One: Fluid Velocity Profiling and Numerical Analysis of Laminar Pipe Flow

This section of the project focuses on the relationship between fluid velocity and radial position within a pipe under laminar flow conditions. Using the Hagen-Poiseuille equation, engineers are able to determine the theoretical velocity profile of air moving through a cylindrical pipe. In this study, experimental data was collected using a hot-wire anemometer, which measures air velocity at various radial positions. However, due to the inherent electronic noise and other physical disturbances in the system, these measurements are not expected to perfectly match theoretical predictions.

The velocity data retrieved from the experiment was encoded in a fixed-point binary format using a legacy 13-bit system—8 bits for the integer portion and 5 bits for the fractional part. To interpret these values, a custom binary-to-decimal conversion process was implemented without relying on built-in MATLAB functions. This allowed for the comparison of experimental and theoretical results by calculating the absolute relative true error at each radial position. Visualizing the theoretical velocity profile against the experimental data not only highlights the accuracy of the measurement system but also provides insight into fluid behaviour under real-world conditions.

Chapter 1: Extracting data, converting it into base-10 and finding error

Before converting the given data presented in Table 1 to base-10, theoretical velocities were found at each radial position using the Hagen-Poiseuille equation as described in **Figure 1**.

```

r = [-0.2;-0.16; -0.12; -0.08 ;-0.04; 0; 0.04 ;0.08; 0.12; 0.16; 0.20];
V_theoretical = [];
delta_p = 5;
R = 0.2;
L = 20;
Dyn_visc = 1.813*10^(-5);
for i = 1:length(r)
    V_theoretical(i,1) = delta_p * ((R^2)/(4*Dyn_visc*L)) * (1-((r(i)^2)/(R^2)));
end

```

Figure 1: Code for Calculating Theoretical Velocities

base-10 format. The data was originally collected in a fixed-point 13-bit binary format—8 bits allocated for the integer portion and 5 bits for the fractional component. Unlike conventional binary conversion, built-in MATLAB functions such as bin2dec were not permitted, necessitating the development of a custom decoding algorithm. The raw data is presented in Table 1.

The key to an accurate conversion lies in preserving the correct order of operations and properly handling the fractional bits. Each binary string was split into two parts: the first 8 bits representing the integer portion, and the last 5 bits representing the fractional part. Each bit's positional value was carefully multiplied by its corresponding power of two—positive exponents for the integer side and negative exponents for the fractional side.

Table 1: Velocity Vs Radial position, Raw Binary Data

Radial Position, r[m]	Binary Velocity Data, VelocityB
-0.2	00000001.00111
-0.16	00111001.00010
-0.12	01010101.11010
-0.08	01110100.00111
-0.04	10000110.10100
0.00	10001010.01011
0.04	10000000.11000
0.08	01110001.11110
0.12	01010111.10111
0.16	00110111.10001
0.20	00000011.01110

An important observation was that the binary strings represented positive velocity values, and thus no sign-bit complications were present. However, ensuring the precision of the fractional

```

VelocityB = load('Ch1_Velocity_Binary.txt');
A = size(VelocityB);
Velocity_10 = [];
V = 0;
for i = 1:A(1)
    n = 7;
    V = 0;
    for j = 1:A(2)
        V = V + 2^n * VelocityB(i,j);
        n = n-1;
    end
    Velocity_10(i,1) = V;
end

```

Figure 2: Binary to Base-10 Conversion

part required meticulous bit-wise processing. For each binary number, the fractional part was processed by summing each bit times 2^{-n} , where n is the position of the bit starting from 1. The integer and fractional results were then summed to form the final base-10 velocity.

This decoding process was implemented in MATLAB using loops and logical operations to fully reconstruct each experimental velocity value in decimal form. The resulting base-10 values were later used for error analysis and comparison

against theoretical values. The MATLAB code to convert the raw data to base 10 is provided in **Figure 2**.

Comparison between the theoretical values and the values extracted from the binary data was also visualized using a graph, which is provided in **Figure 3**. The theoretical velocity profile was calculated using the Hagen-Poiseuille equation over a finely spaced radial range from -0.2 m to 0.2 m , with increments of 0.001 meters . This resulted in a smooth parabolic curve representing ideal laminar flow conditions in the pipe.

The experimental velocity values, obtained by converting binary data into base-10 format, were plotted at their corresponding radial positions to form a discrete experimental profile. Both curves were displayed on the same axes to allow for visual comparison.

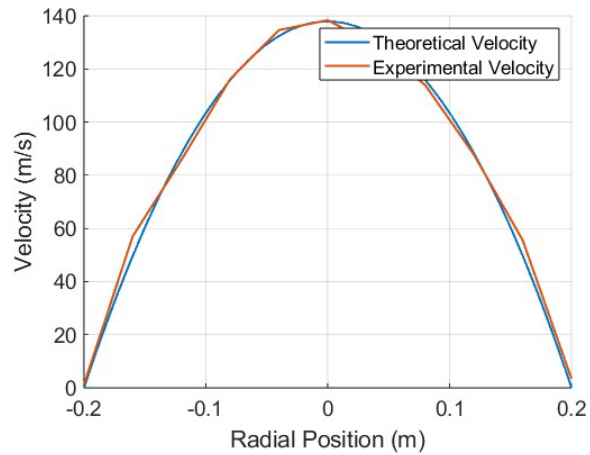


Figure 3: Theoretical and Experimental Velocity Vs Radial Pos.

Once the experimental velocity values were successfully converted from binary to decimal form, the next step involved quantifying the deviation of these values from the theoretical predictions. This was achieved by calculating the **absolute relative true error** at each radial position along the pipe. In this analysis, the velocity predicted by the Hagen-Poiseuille equation was treated as the true value, while the experimentally obtained velocity from the hot-wire anemometer was considered the approximate value.

The formula used to compute the absolute relative true error (as a percentage) is given by:

$$\text{Absolute Relative True Error}(\%) = \left| \frac{\text{Theoretical Velocity} - \text{Recorded Velocity}}{\text{Theoretical Velocity}} \right| * 100$$

In MATLAB, this computation was implemented using a loop that iterated through each pair of theoretical and experimental velocity values. The result was stored in a vector E_t , which represents the error as a function of radial position. The corresponding MATLAB code is shown in **Figure 4** below.

```
Et = [];
E_t = 0;
for i = 1:length(Velocity_10)
    E_t = abs((V_theoretical(i)-Velocity_10(i))/(V_theoretical(i))*100);
    Et(i,1) = E_t;
end
```

Figure 4: Code for Absolute Relative True Error

To analyse the accuracy of the experimental velocity data, the absolute relative true error was plotted as a function of radial position, shown in **Figure 5**. This plot provides a clear visual representation of how the error between theoretical and experimental values varies across the pipe's cross-section. Note: 2 extreme values were not plotted as both of them are infinity.

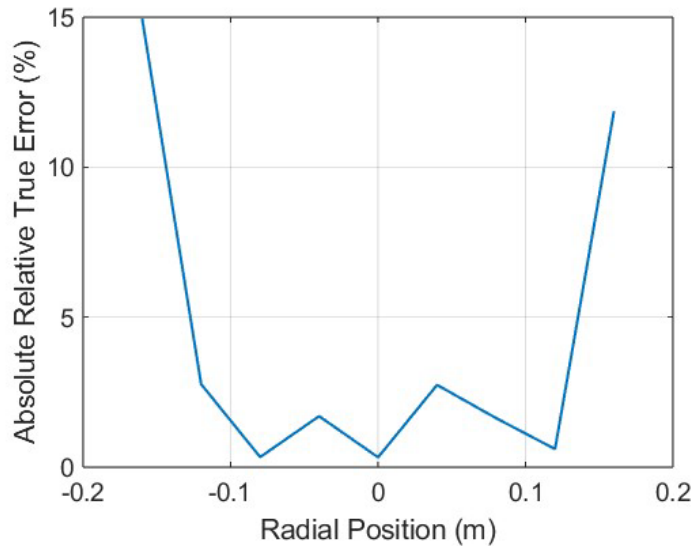


Figure 5: Absolute Relative True Error Vs Radial Position

To summarize the findings, a table was created containing four key columns: the radial position (in meters), the theoretical velocity calculated using the Hagen-Poiseuille equation, the experimentally obtained velocity values (converted to base-10), and the absolute relative true error (as a percentage). **Table 2** enables a clear side-by-side comparison between the ideal and measured data. For any data points that deviated drastically from the ideal was labelled as N/A to reflect infinity.

Table 2: Values at their respective radial position

Radial Position	Theoretical Velocity	Experimental Velocity	Abs. Rel. True Error
-0.2	0	1.218750	N/A
-0.16	49.6414782129068	57.06250000000000	14.9492361111110
-0.12	88.2515168229455	85.81250000000000	2.76371093750006
-0.08	115.830115830116	116.21875000000000	0.33552083333328
-0.04	132.377275234418	134.62500000000000	1.69796874999997
0.00	137.892995035852	138.34375000000000	0.326887499999957
0.04	132.377275234418	128.75000000000000	2.74010416666670
0.08	115.830115830116	113.93750000000000	1.633958333333338
0.12	88.2515168229455	87.71875000000000	0.603691406250060
0.16	49.6414782129068	55.53125000000000	11.86461805555555
0.20	0	3.4375000000000000	N/A

Questions For Chapter 1:

- The mismatch between experimental and theoretical velocity values—particularly near the pipe walls—can be attributed to multiple factors. First, in the theoretical model, we assume a no-slip condition at the pipe walls, meaning the fluid velocity is exactly zero at $r = \pm R$. However, in practice, it is challenging to measure velocity precisely at the wall due to sensor

limitations and physical constraints. As a result, the experimental values near the wall may show small non-zero velocities, deviating from the ideal model.

Additionally, the theoretical velocity profile is derived under several simplifying assumptions, including steady, incompressible, fully developed, laminar flow, and negligible entrance effects. In real experimental setups, some of these assumptions may not hold exactly. For example, there may be minor turbulence, imperfectly developed flow, or fluctuations in pressure and temperature. These unmodeled effects introduce discrepancies between the measured and theoretical values, especially at regions like the wall where velocity gradients are steep and highly sensitive to boundary effects.

b. The relative true error increases significantly near the pipe walls because the theoretical velocity approaches zero at $r = \pm R$, due to the no-slip boundary condition. Since the relative true error is calculated by dividing the difference between the theoretical and experimental values by the theoretical value, we encounter a mathematical issue:

$$\text{Absolute Relative True Error}(\%) = \left| \frac{\text{Theoretical Velocity} - \text{Recorded Velocity}}{\text{Theoretical Velocity}} \right| * 100$$

As the denominator $V_{\text{theoretical}}$ approaches zero, even small differences in the numerator (i.e., experimental deviation) cause the error to spike dramatically. In some cases, this leads to an undefined or infinite error, which explains why the relative error graph increases sharply near the wall and why some values may be marked as N/A in the error table.

c. Using a fixed-point binary representation with limited bits introduces round-off error because not all real numbers can be exactly represented. In this project, each experimental velocity is stored using 8 bits for the integer part and 5 bits for the fractional part, meaning we can only represent numbers in discrete steps of $2^{-5} = 0.03125$. Any velocity value that falls between these discrete levels must be rounded to the nearest available value, causing a loss in precision.

This rounding effect is a type of round-off error, which becomes more pronounced when the step size (least count) is relatively large — as is the case in this fixed-point system. These small inaccuracies accumulate and contribute to discrepancies between theoretical and experimental results.

This concept is closely related to machine epsilon (ϵ_{mach}), which is defined as the smallest positive number such that:

$$1 + \epsilon_{\text{mach}} > 1$$

In other words, machine epsilon is the smallest difference between two defined numbers in a system is known as the machine epsilon. For our fixed-point system with only 5 bits for the fractional component, the effective machine epsilon is much larger — about 0.03125. This larger epsilon means less precision, making round-off error more significant and one of the contributing factors to the difference between theoretical and recorded velocity values.

Chapter 2: Numerical Differentiation

In this section, we implement numerical differentiation to calculate shear stress across the radius of the pipe based on the experimental velocity values derived in the earlier phase of the project. Shear stress in a fluid flowing through a pipe is defined by the equation:

$$\tau = \mu \frac{\partial u}{\partial r}$$

where τ is the shear stress, μ is the dynamic viscosity of the fluid, and $\frac{\partial u}{\partial r}$ is the velocity gradient with respect to the radial position r . For this experiment, the air has a dynamic viscosity of

$$\mu = 1.813 \times 10^{-5} \text{ Pa} \cdot \text{s at } 20^\circ\text{C}.$$

To numerically estimate the velocity gradient, finite difference methods were employed:

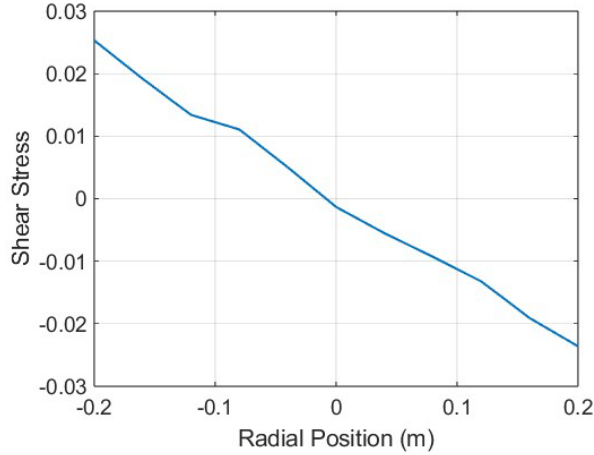


Figure 6: Shear Stress Vs Radial Position

- The forward difference method was used at the first data point.
- The backward difference method was used at the last data point.
- The central difference method was used at all interior points.

Whenever possible, the central difference method was preferred, as it generally provides the most accurate estimate of the derivative. However, it requires data points on both sides of the location being evaluated. This limitation means that

forward or backward differences must be used at the edges of the dataset, where neighbouring points are not available on one side. This approach ensures accurate and stable gradient estimation, especially in regions where data is symmetrically spaced. The results from this method is provided in **Figure 6**. The code was designed to be generalizable for any number of radial data points, assuming uniform spacing between them (i.e., a constant h). The code is presented in **Figure 7**.

```
shear = [];  
for i = 1:length(r)  
    if i == 1  
        h_FDD = r(i+1)-r(i);  
        shear(i,1) = Dyn_visc * ((Velocity_10(i+1)-Velocity_10(i))/(r(i+1)-r(i)));  
    elseif i ~= 1 && i ~= length(r)  
        h_CDD = r(i+1) - r(i-1);  
        shear(i,1) = Dyn_visc * ((Velocity_10(i+1)-Velocity_10(i-1))/(h_CDD));  
    elseif i == length(r)  
        h_BDD = r(i) - r(i-1);  
        shear(i,1) = Dyn_visc * ((Velocity_10(i)-Velocity_10(i-1))/h_BDD);  
    end  
end
```

Figure 7: Code to Numerically Differentiate for Shear Stress

Question For Chapter 2:

Based on the shear stress graph, the magnitude of shear stress is largest at the pipe walls, which correspond to the radial positions near ± 0.2 meters. This behavior is consistent with theoretical expectations for fully developed laminar flow in a pipe, where the shear stress is zero at the centerline ($r = 0$) and increases linearly in magnitude toward the walls, reaching its maximum at the pipe's inner surface. The sign of the shear stress flips across the centerline due to the symmetry of the velocity profile, but the maximum absolute value (magnitude) occurs at the boundaries. Thus, the largest shear stress value by magnitude is seen at $r = -0.2m$

Chapter 3: Root finding

Lastly, to conclude part one, we implement the Bisection Method, a root-finding technique, to determine the radial positions in the pipe where specific velocities occur. Earlier in the project, the velocity profile as a function of radial position was introduced and is given by:

$$u(r) = \frac{\Delta P R^2}{4\mu L} \left(1 - \frac{r^2}{R^2} \right)$$

This equation models laminar flow through a circular pipe and describes how fluid velocity decreases parabolically from the centre of the pipe toward the walls. Using this expression, we can now work backwards given a velocity value, we want to determine the radial location(s) at which this velocity occurs.

To achieve this, we applied the Bisection Method. This numerical approach finds roots of equations by repeatedly halving an interval and selecting subintervals in which the function changes sign. In our case, we aim to solve:

$$f(r) = u(r) - V_{find} = 0$$

for a given V_{find} which is the velocity we want to find. The method continues until the absolute relative approximate error falls below 0.00005%, using while loop ensuring a highly accurate estimate of the radial position. The code has been designed to take input from the user for the velocity to find and keeps track of the number of iterations and the velocity found at each iteration. The code also takes input for the initial guess but only for the higher guess as the lower guess stays the same as 0.

For this task, three target velocities were tested: 62.1 m/s, 17.3 m/s, and 0.81 m/s. The function iteratively narrows down the search range and converges in on the correct radial position. The velocity equation's parabolic nature ensures that the same velocity can occur at both positive and negative radial positions, symmetrically located about the centerline of the pipe.

```

V_find = input('Enter Velocity to be found = ');
r_low = 0;
r_high = input('Enter Initial High Guess = ');
r_values = [];

V_low = delta_p * ((R^2)/(4*Dyn_visc*L)) * (1-((r_low^2)/(R^2))) - V_find; % Subtracting V_find to make it equal to zero
V_high = delta_p * ((R^2)/(4*Dyn_visc*L)) * (1-((r_high^2)/(R^2))) - V_find; % Subtracting V_find to make it equal to zero

if V_low * V_high > 0
    error('No root exists in the given range. Change your initial guess.');
```

```

end

Prev_appx = 0;
abs_Rel_appx = 500;
n = [];
N = 0;
V_disp = [];

while abs_Rel_appx > 0.00005
    N = N + 1;
    n = [n; N];
    r_mean = (r_low + r_high) / 2;
    r_values = [r_values; r_mean];
    % Computing function value at midpoint
    V_mean = delta_p * ((R^2)/(4*Dyn_visc*L)) * (1-((r_mean^2)/(R^2))) - V_find; % Subtracting V_find to make it equal to zero
    V_disp = [V_disp; V_mean+V_find];
    Curr_appx = r_mean;
    if Prev_appx ~= 0
        abs_Rel_appx = abs((Curr_appx - Prev_appx) / Curr_appx) * 100;
    end
    Prev_appx = Curr_appx; % Update for next iteration
    if V_low * V_mean < 0
        r_high = r_mean;
        V_high = V_mean;
    else
        r_low = r_mean;
        V_low = V_mean;
    end
end
end

```

Figure 8: Code to find Root Using Bisection method

The MATLAB code for this procedure is shown in **Figure 8**, and the iteration data and convergence results, is shown in **Table 3**. One of the more subtle challenges in coding this method was determining which bound to replace in each iteration. This was addressed by analysing the sign of the function value at the midpoint relative to the signs at the current bounds—a classic approach in numerical root-finding. This technique proved effective in locating the radial positions corresponding to specific velocity values and demonstrated the usefulness of numerical methods in solving real-world fluid mechanics problems where analytical inversion of equations is impractical.

Table 3: Results of Bisection Method

Velocity to Find (m/s)	Radial Position (m)	Number of Iterations	Velocity at the found Radial Pos. (m/s)
62.1	0.1483	22	62.100000
17.3	0.1870	22	17.300000
0.81	0.1994	21	0.81010000

Questions for Chapter 3:

The bisection method is a simple and reliable root-finding technique based on the Intermediate Value Theorem. It works by repeatedly halving an interval $[a, b]$ between which a function changes sign, ensuring that a root exists within the interval.

Advantages:

- **Guaranteed Convergence:** If the function is continuous and the initial guesses bracket a root (i.e., $f(a) \cdot f(b) < 0$), the method is guaranteed to converge.
- **Interval Reduction:** Every iteration cuts the interval in half, consistently bringing us closer to the root. This provides a clear way to bound the true error, which can never exceed half the current interval length. This makes it easy to estimate and control the maximum possible error.
- **Simplicity:** The algorithm is straightforward to implement and understand.

Disadvantages:

- **Slow Convergence:** The convergence rate is linear and relatively slow compared to other methods. It may require many iterations for high precision.
- **Requires Bracketing:** You must know two initial values that bracket the root, which is not always easy to find.
- **Inefficient for Non-Sign-Changing Roots:** If the function does not change sign or has multiple roots in the interval, the method may fail or converge to the wrong root.

a. Describe why one may choose Newton-Raphson over bisection method.\

One may choose the Newton-Raphson method over bisection when faster convergence is desired and additional information about the function is available. Unlike bisection, Newton-Raphson is an open method that uses the function's derivative to estimate the root:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Reasons to prefer Newton-Raphson:

- **Quadratic Convergence:** Near the root, the method converges much faster than bisection—typically doubling the number of correct digits per iteration.
- **No Bracketing Required:** It only needs a single initial guess and does not require the root to be bracketed.
- **Efficient for Smooth Functions:** When the function is differentiable and well-behaved near the root, Newton-Raphson is much more computationally efficient. However, Newton-Raphson does have limitations—it can diverge if the initial guess is poor, and it requires calculation of the derivative, which is not always practical. Nonetheless, when speed and efficiency are important and the function is suitable, Newton-Raphson is often preferred.

Part two: Computational Heat Transfer Analysis Through Composite Materials

In this part of the project, we shift our focus from fluid flow to heat transfer. We begin by visualizing a simple scenario: a series of metal plates, each made from different materials and connected end-to-end. One side is heated while the other remains cool, creating a natural flow of thermal energy through conduction. The setup may appear simple, but it allows us to explore a range of physical principles using computational tools.

Key variables in this model include the temperature at the junctions (called *nodes*), the length and cross-sectional area of each metal segment, and each material's thermal conductivity—essentially how good it is at conducting heat. These properties come together to determine the *thermal resistance*, which tells us how much each segment resists the flow of heat. The higher the conductivity or the larger the area, the easier heat flows; the longer the material, the more it resists.

From this, we calculate the thermal resistance using the formula:

$$R = \frac{L}{k * A}$$

where R is the thermal resistance, L is the length of the plate, k is the thermal conductivity of the respective plate and A is the cross-sectional area of the plate.

Throughout this section, we'll use these principles and computational methods to simulate how heat flows through composite materials. The goal is not just to understand heat transfer conceptually, but to implement and solve real-world problems using programming and numerical techniques. This approach makes the abstract more tangible and gives us the tools to analyse complex systems that can't be easily solved by hand.

Chapter 4: Linear System of Equations

In this section of the project, we analyse the steady-state heat conduction through a series of four different metal blocks arranged in direct contact, as shown in Figure 4. Each block has a different thermal conductivity and length, and heat flows from the hot end ($T_0 = 800^\circ\text{C}$) to the cold end ($T_4 = 0^\circ\text{C}$). Using the physical and thermal properties provided, our goal is to determine the temperatures at the three internal nodes: T_1 , T_2 , and T_3 .

The thermal resistance for each segment is calculated using the formula:

$$R_i = \frac{L_i}{k_i * A}$$

Once these resistances are determined, we can apply conservation of energy at each interior node to generate three equations involving the unknown temperatures. These equations are derived from the principle that, at steady state, the rate of heat entering a node must equal the rate of heat leaving it. The resulting system is:

$$\begin{aligned}(R_1 + R_2)T_1 - R_1T_2 &= R_2T_0 \\ -R_3T_1 + (R_2 + R_3)T_2 - R_2T_3 &= 0 \\ -R_4T_2 + (R_3 + R_4)T_3 &= R_3T_4\end{aligned}$$

```

T_0 = 800;
T_4 = 0;
L1 = 0.03;
L2 = 0.02;
L3 = 0.01;
L4 = 0.01;
k1 = 400;
k2 = 240;
k3 = 16;
k4 = 10;
Length = [L1 L2 L3 L4];
A = 0.01;
k = [k1 k2 k3 k4];
R1 = [];
for i = 1:length(Length)
    R_val = (Length(i))/(k(i)*A);
    R1(1,i) = [R_val];
end
A = [R1(1,1)+R1(1,2) -R1(1,1) 0; -R1(1,3) R1(1,2)+R1(1,3) -R1(1,2); 0 -R1(1,4) R1(1,3)+R1(1,4)];
B = [T_0*R1(1,2); 0; R1(1,3)*T_4];
x = A\B;
T_1 = x(1,1);
T_2 = x(2,1);
T_3 = x(3,1);
L_plot = 0;
for i = 1:length(Length)
    L_plot(end+1) = L_plot(end) + Length(i) ;
end
T = [T_0 T_1 T_2 T_3 T_4];
T = round(T,6,'significant');
figure(4)
plot(L_plot, T, 'o', MarkerFaceColor= 'b', MarkerEdgeColor = 'b',LineWidth=2);
xlim([0 0.08]);
ylim([0 1000]);
xlabel('Length (m)', 'FontSize',14);
ylabel('Temperature (C)', 'FontSize',14);
title('Length vs Temperature');
legend('True Temperature Values', 'Location', 'northeast');
set(gca, 'FontSize',14);
grid on;

```

Figure 9: Code to Solve for the Temperatures

To solve this system efficiently, the equations were rearranged so that all terms involving the unknowns T_1 , T_2 , and T_3 appear on the left-hand side, allowing us to express the system in the matrix form: $Ax = B$, Where,

$$\underbrace{\begin{bmatrix} R_1 + R_2 & -R_1 & 0 \\ -R_3 & R_2 + R_3 & -R_2 \\ 0 & -R_4 & R_3 + R_4 \end{bmatrix}}_A \times \underbrace{\begin{bmatrix} T_1 \\ T_2 \\ T_3 \end{bmatrix}}_x = \underbrace{\begin{bmatrix} T_0 * R_2 \\ 0 \\ R_3 * T_4 \end{bmatrix}}_B$$

Here, x is the vector of unknown temperatures, A is a 3x3 matrix containing the resistance coefficients, and B is the known temperature influence vector based on the boundary conditions. This matrix representation simplifies the process of solving the system using MATLAB this code can be referred to in **Figure 9**, particularly with the backslash operator ($x=A \setminus B$) which internally handles matrix decomposition and computation. This method allows for flexible analysis: by changing the input values for k , L , and A , the system can be re-solved for entirely new material setups. The found Temperature values were recorded and are presented in **Table 4**.

Table 4: Found Temperature Values with 4 significant Figures represented with chopping

Parameter	Values
T_1	766.3

T_2	728.9
T_3	448.5

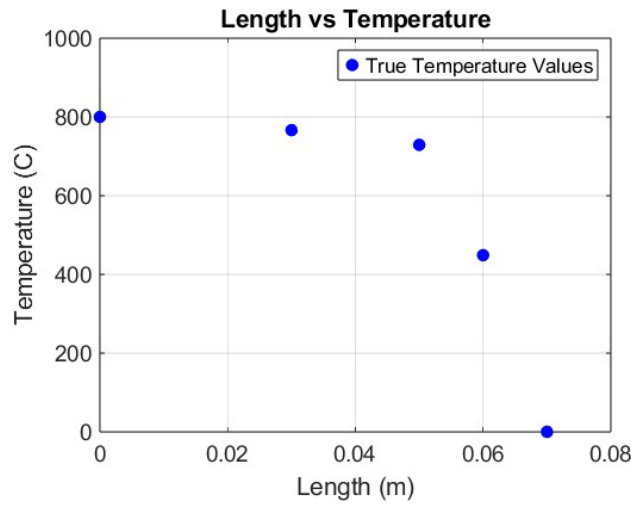


Figure 10: Length Vs Found Temperature

reflects the varying lengths of each material segment. This allows us to see not only the temperature values, but also where sharper or more gradual temperature drops occur—typically corresponding to differences in thermal resistance.

Questions for chapter 4:

To solve a linear system of equations written in matrix form: $Ax = B$ where:

- A is the coefficient matrix (e.g., based on thermal resistances),
- x is the vector of unknowns (e.g., $[T_1, T_2, T_3]$),
- B is the constants vector,

we can use the following three methods manually.

Given a system:

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}}_A \times \underbrace{\begin{bmatrix} T_1 \\ T_2 \\ T_3 \end{bmatrix}}_x = \underbrace{\begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}}_B$$

Gaussian Elimination: The goal is to convert the matrix A to an upper triangular matrix using row operations, then apply back substitution.

1. Forward Elimination

Use row operations to zero out the sub-diagonal elements (a_{21}, a_{31}, a_{32}):

- Eliminate a_{21} :

$$Row'_2 \leftarrow Row_2 - \left(\frac{a_{21}}{a_{11}}\right) \cdot Row_1$$

- Eliminate a_{31} , then a_{32} using similar multipliers.

2. Back Substitution

Solve the upper triangular system starting from the last row:

$$T_3 = \frac{b'_3}{a'_{33}}$$

$$T_2 = \frac{b'_2 - a'_{23}T_3}{a'_{22}}$$

To visualize how thermal energy distributes through the different materials, the calculated temperatures at each node—including the boundary temperatures T_0 and T_4 —were plotted in **Figure 10** as a function of position along the length of the composite wall. This representation helps illustrate the gradual decrease in temperature from the hot end to the cold end as heat flows through the system.

Each data point corresponds to a physical location where two materials meet. The spacing between these points

$$T_1 = \frac{b'_1 - a_{12}T_2 - a_{13}T_3}{a_{11}}$$

LU Decomposition: Factor matrix A into a lower triangular matrix L and an upper triangular matrix U , then solve the system in two steps. Where the upper triangular matrix U is obtained as the final result of the forward elimination process used in Gaussian elimination and the lower triangular matrix L is formed by collecting all the multipliers used during elimination to eliminate entries below the main diagonal. These multipliers reflect how each row was adjusted during the process.

$$A = LU$$

1. Decompose A
 - Perform Gaussian elimination and record the multipliers in a lower triangular matrix L with 1s on the diagonal
 - The result of the elimination forms matrix U .
2. Define matrix Z
 - After Decomposing Matrix A , the equation looks like: $\underbrace{LU}_A \times x = B$
 - Assume $Z = Ux$, where $Z = \begin{bmatrix} Z_1 \\ Z_2 \\ Z_3 \end{bmatrix}$
 - Rewriting the equation: $LZ = B$
 - Solve for Z using forward substitution, which is the same as back substitution, but we start from Z_1 this time.
 - Now that we know Z , we can solve for x . But here when solving for x we have to use back substitution.

$$Ux = Z$$

Partial Pivoting: Partial pivoting is an enhancement to Gaussian elimination that increases numerical accuracy and prevents division by small or zero values. The idea is to rearrange the rows of the matrix during the elimination process to ensure that the pivot element (the diagonal entry used for elimination) is the largest possible (in magnitude) in its column.

1. Reorder the rows so that the largest (in magnitude) element in the current column is placed in the pivot position (on the diagonal), which helps maintain numerical stability during elimination and we do not divide by zero. If $a_{21} > a_{11}$ then these two elements shift places.

Thus $A = \begin{bmatrix} a_{21} & a_{12} & a_{13} \\ a_{11} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$. This check has to be done for every element in the lower triangle.

2. Proceed with Gaussian elimination using the new row arrangement.

Chapter 5: Interpolation and Splines

This part of the project is divided into two main parts, with the first focusing on estimating temperatures between the known node values calculated previously. While temperature change in a uniform material is typically linear, the presence of varying thermal conductivities and segment lengths between materials introduces non-linearities. As a result, interpolation techniques provide a valuable way to estimate temperature at any point along the composite wall with improved accuracy.

Interpolation

Interpolation is a mathematical method used to estimate unknown values that fall between known data points. By fitting a polynomial curve to the known data, we can evaluate the function at intermediate points where measurements were not taken. This is especially useful in thermal analysis when we want to understand the temperature behavior continuously across materials, not just at discrete nodes.

```
A = [0 0 0 1; 0.03^4 0.03^3 0.03^2 0.03^1 1; 0.05^4 0.05^3 0.05^2 0.05^1 1; 0.06^4 0.06^3 0.06^2 0.06^1 1; 0.07^4 0.07^3 0.07^2 0.07^1 1];
B = [T(1,1); T(1,2); T(1,3); T(1,4); T(1,5)];
I = A\B;
T_interpol = [];
for i = 1:length(L_plot)
    T_interpol(1,i) = I(1)*L_plot(i)^4 + I(2)*L_plot(i)^3 + I(3)*L_plot(i)^2 + I(4)*L_plot(i)^1 + I(5)*L_plot(i)^0 ;
end
figure(5)
plot(L_plot, T_interpol, 'o', 'MarkerFaceColor', 'k', 'MarkerEdgeColor', 'k', 'LineWidth', 1);
xlim([0 0.08]);
ylim([0 1000]);
xlabel('Length (m)', 'FontSize',14);
ylabel('Temperature (C)', 'FontSize',14);
title('Length vs Temperature');
set(gca, 'FontSize',14);
grid on;
legend('Interpolated Points', 'Location', 'northeast');
```

Figure 11: Code for interpolation of temperature

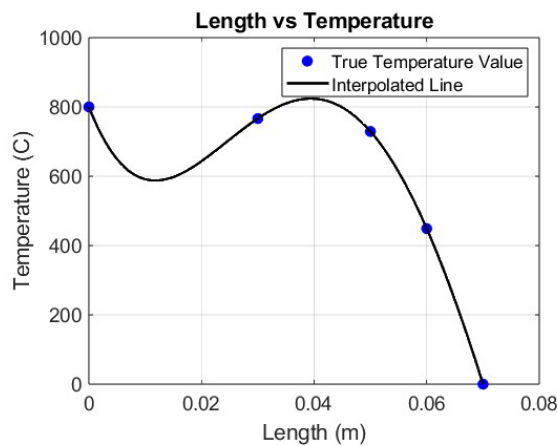


Figure 12: Interpolated Temperature at the specific Locations

In this case, we have five known temperature values at $x = 0m$, $x = 0.03m$, $x = 0.05m$,

$x = 0.06m$ and $x = 0.07m$. To construct a polynomial that passes exactly through all five of these points, we would need a 4th-order interpolant. This is because a polynomial of degree n can pass through $n + 1$ data points. So, with five known temperatures a 4th-order polynomial (degree 4) is required. The matrix containing all the equations used to interpolate are provided below. To solve the matrix same method is used when we were solving for the temperatures, which is $I = A \setminus B$, where A is a

(5×5) matrix containing all the coefficients and B is (5×1) matrix having all the known temperatures from the previous part. The code for this part can be referred to in **Figure 11**. After solving the matrix and getting the coefficients the solved interpolant equation was used to calculate and plot the temperature values provided in **Figure 12** at the range of x values on which we already know temperature from the previous part.

$$f(x) = a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$$

$$\underbrace{\begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0.03^4 & 0.03^3 & 0.03^2 & 0.03 & 1 \\ 0.05^4 & 0.05^3 & 0.05^2 & 0.05 & 1 \\ 0.06^4 & 0.06^3 & 0.06^2 & 0.06 & 1 \\ 0.07^4 & 0.07^3 & 0.07^2 & 0.07 & 1 \end{bmatrix}}_A \times \underbrace{\begin{bmatrix} a_4 \\ a_3 \\ a_2 \\ a_1 \\ a_0 \end{bmatrix}}_x = \underbrace{\begin{bmatrix} 800 \\ 766.355 \\ 728.972 \\ 448.598 \\ 0 \end{bmatrix}}_B$$

Rough work:

We have 5 data points. ($n = 5$)

We already know that to interpolate n data points we need $n - 1^{th}$ order of interpolant. In our case we have 5 data points, which requires a 4^{th} order interpolant.

To find the equations we relate every temperature value to the corresponding length using the interpolant equation, as shown below.

$$\begin{aligned}800 &= a_4(0)^4 + a_3(0)^3 + a_2(0)^2 + a_1(0) + a_0 \\766.355 &= a_4(0.03)^4 + a_3(0.03)^3 + a_2(0.03)^2 + a_1(0.03) + a_0 \\728.972 &= a_4(0.05)^4 + a_3(0.05)^3 + a_2(0.05)^2 + a_1(0.05) + a_0 \\448.598 &= a_4(0.06)^4 + a_3(0.06)^3 + a_2(0.06)^2 + a_1(0.06) + a_0 \\0 &= a_4(0.07)^4 + a_3(0.07)^3 + a_2(0.07)^2 + a_1(0.07) + a_0\end{aligned}$$

This higher-order polynomial allows us to model potential curvature in the temperature profile that might result from the varying thermal resistances between materials. While higher-order interpolation increases accuracy within the known range, it's important to remember that such methods can become unstable or misleading outside the bounds of the known data (i.e., extrapolation).

Questions for Chapter 5 Interpolation:

a. According to me, no—it's not a great representation of the physical system. Even though the interpolated curve technically passes through all the temperature data points, the curve itself exhibits noticeable wiggles and overshoots that don't make physical sense. The system being modelled—steady-state heat conduction through solid plates—is known to be linear within each material. So, the temperature should vary smoothly and predictably between nodes which has a gradual decline. The high-order polynomial used in the interpolation tries too hard to fit every point exactly and ends up creating unrealistic peaks and dips. In the real world, temperature doesn't spike like that unless energy is being added or lost from the system, which isn't the case here. From a physical standpoint, this curve would raise red flags—it implies energy is being added or removed in ways that violate the basic laws of conduction. So, while it's mathematically accurate in the interpolation sense, it's not a faithful representation of the actual thermal behavior of the system.

b. The unusual shape of the interpolated curve is due to the use of a high-order polynomial interpolant—specifically a 4th-degree polynomial across 5 nodes. As discussed in class, when we force a single polynomial to pass through many points, the interpolant begins to oscillate wildly, especially near the endpoints. These oscillations are purely mathematical artifacts—they aren't driven by the underlying physics.

Splines:

For next part, to better understand how temperature varies continuously between the known node points, we now introduce quadratic splines as an interpolation method. Unlike single high-degree polynomials, which may oscillate excessively between data points, splines use piecewise polynomials that ensure smooth transitions and better stability across intervals.

In this part of the analysis, quadratic splines are used to construct a series of parabolic segments that connect each pair of adjacent temperature data points. For n data points, $n - 1$ spline segments are needed—so with five known temperatures, we require four separate quadratic spline equations to describe the temperature distribution across the full length.

Each spline is a second-degree polynomial of the form:

$$f(x) = a_i x^2 + b_i x + c_i$$

```
x1 = L_plot(1,1);
x2 = L_plot(1,2);
x3 = L_plot(1,3);
x4 = L_plot(1,4);
x5 = L_plot(1,5);
b1 = T(1,1);
b2 = T(1,2);
b3 = T(1,3);
b4 = T(1,4);
b5 = T(1,5);
A = [x1^2 x1 1 0 0 0 0 0 0 0 0 0;
     x2^2 x2 1 0 0 0 0 0 0 0 0 0;
     0 0 0 x2^2 x2 1 0 0 0 0 0 0;
     0 0 0 x3^2 x3 1 0 0 0 0 0 0;
     0 0 0 0 0 0 x3^2 x3 1 0 0 0;
     0 0 0 0 0 0 0 x4^2 x4 1 0 0 0;
     0 0 0 0 0 0 0 0 0 x4^2 x4 1;
     0 0 0 0 0 0 0 0 0 x5^2 x5 1;
     2*x2 1 0 -2*x2 -1 0 0 0 0 0 0 0;
     0 0 0 2*x3 1 0 -2*x3 -1 0 0 0 0;
     0 0 0 0 0 0 2*x4 1 0 -2*x4 -1 0;
     0 0 0 0 0 0 0 0 0 0 1 0];
B = [b1; b2; b2; b3; b3; b4; b4; b5; 0; 0; 0; 0];
x = A\B;
% x = [a1; b1; c1; a2; b2; c2; a3; b3; c3; a4; b4; c4]
hold on
x_vals = L_plot(1, :);
y_vals = T(1, :);
for i = 1:4
    j = 3*i - 2; % indexing for coefficients
    a = x(j);
    b = x(j+1);
    c = x(j+2);
    % Defining intervals for current spline as intervals are not equally spaced
    x_range = linspace(x_vals(i), x_vals(i+1), 10);
    yval = a * x_range.^2 + b * x_range + c;
    plot(x_range, yval, 'Color', 'r', 'LineWidth', 2)
    set(gca, 'FontSize', 14);
end
legend('Interpolated Points', 'Spline fit')
```

Figure 13: Code for Spline fitting

boundary condition. In this analysis, since all intervals are equal, we assume the last spline is linear.

For four spline segments, this results in twelve total equations (three for each spline: two for endpoints, one for continuity in slope). The matrix system was implemented in MATLAB to solve for the spline coefficients, which were then used to construct and plot each individual spline segment. The MATLAB code used for this process, along with the resulting spline plot plotted on top of **Figure 12**, is presented in **Figure 13** and **Figure 14**, respectively.

Equations: 1-8 Equations are produced by equating the corresponding temperature values

$$1. a_1 x_1^2 + b_1 x_1 + c_1 = 800$$

$$2. a_1 x_2^2 + b_1 x_2 + c_1 = 766.3551$$

$$3. a_2 x_2^2 + b_2 x_2 + c_2 = 766.3551$$

$$4. a_2 x_3^2 + b_2 x_3 + c_2 = 728.9720$$

$$5. a_3 x_3^2 + b_3 x_3 + c_3 = 728.9720$$

$$6. a_3 x_4^2 + b_3 x_4 + c_3 = 448.5891$$

$$7. a_4 x_4^2 + b_4 x_4 + c_4 = 448.5891$$

$$8. a_4 x_5^2 + b_4 x_5 + c_4 = 448.5891$$

To uniquely determine the coefficients of these polynomials, a system of equations must be constructed. We need $3p = 3 * 4 = 12$ equations, where $p = \text{no. of segments}$. The following constraints are used to build this system:

1. Continuity at data points: Each spline must pass through its corresponding endpoints.

2. Smoothness: The first derivatives of adjacent splines must be equal at interior points.

3. Boundary condition: One additional equation is required to close the system. Typically, we assume that the first or last spline is linear at the boundary. This is determined by which spline from the first and last has the shorter interval making it a logical choice for applying the

9-11 Equations are produced by equating the derivative of the inner points i.e. x_2, x_3, x_4 .

$$9. \quad 2a_1x_2 + b_1 - 2a_2x_2 - b_2 = 0$$

$$10. \quad 2a_2x_3 + b_2 - 2a_3x_3 - b_3 = 0$$

$$11. \quad 2a_3x_4 + b_3 - 2a_4x_4 - b_4 = 0$$

The 12th Equation comes from assuming the last spline is linear.

$$12. \quad a_4 = 0$$

To solve this system, the equations are written in matrix form:

$$\underbrace{\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 9e^{-4} & 0.03 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 9e^{-4} & 0.03 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.0025 & 0.05 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.0025 & 0.05 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.0036 & 0.06 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.0036 & 0.06 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.0049 & 0.07 & 1 \\ 0.06 & 1 & 0 & -0.06 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.1 & 1 & 0 & -0.1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.12 & 1 & 0 & -0.12 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}}_A \times \underbrace{\begin{bmatrix} a_1 \\ b_1 \\ c_1 \\ a_2 \\ b_2 \\ c_2 \\ a_3 \\ b_3 \\ c_3 \\ a_4 \\ b_4 \\ c_4 \end{bmatrix}}_I = \underbrace{\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}}_B$$

The plot in **Figure 14** illustrates the resulting quadratic spline interpolation across the composite wall. Each coloured segment represents one of the four spline equations connecting adjacent temperature nodes, allowing for a smooth and continuous estimate of the temperature profile along the wall's length.

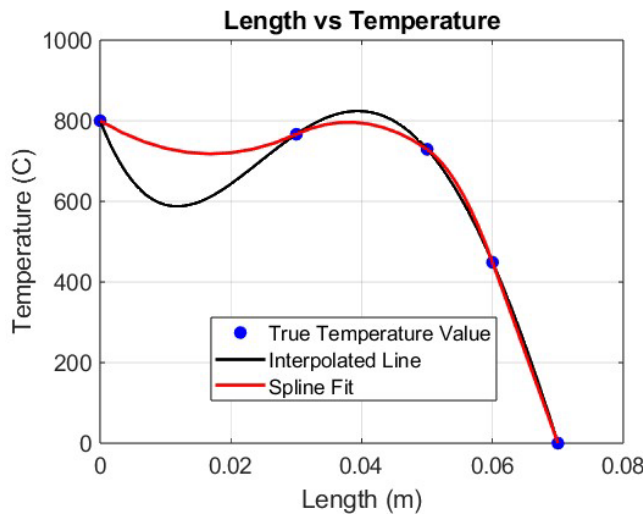


Figure 14: Temperature Vs Length Using Spline

Instead, the curvature of each spline reflects the influence of local material properties. For example, the steeper drop in the final segment (Spline 4) is consistent with a material of lower thermal conductivity, causing a sharper temperature gradient. This spline-based visualization provides a much more accurate and realistic picture of how thermal energy propagates through the multi-material system compared to linear approximations. It also demonstrates the value of computational methods in modelling physical behavior across complex systems.

The black markers indicate the known data points (i.e., the nodal temperatures calculated earlier), while the splines in magenta, green, blue, and red correspond to Spline 1 through Spline 4, respectively. These splines together form a continuous curve that models how temperature changes between the known locations, accounting for material differences and varying thermal resistances.

From the plot, we can observe that temperature does not decrease linearly.

Questions for Chapter 5 Splines:

1. The spline fit is noticeably better than the previous high-order polynomial interpolant. While the interpolated line (black curve) as shown in **Figure 14** shows exaggerated bends and non-physical oscillations between points—especially near the boundaries—the spline (red curve) is smoother and more stable throughout the domain. It still connects all the data points, but it doesn't try to overfit them the way the global polynomial does.

So, to answer the question: I would definitely choose the spline to estimate values between node temperatures. It provides a more physically realistic shape while still honouring the known data. It's more trustworthy, especially in engineering applications where smooth transitions are expected and important.

2. Heat conduction in a single material with constant thermal conductivity should yield a linear temperature gradient. However, in this problem, we're dealing with a composite system—a chain of materials, each with its own thermal conductivity and length. While the temperature profile within each individual material should be linear, the overall system may exhibit slight curvature at the interfaces due to changes in thermal resistance.

Looking at the spline fit, it's clear that the quadratic segments gently follow the expected behavior. They do a good job of capturing the transitions without overshooting or distorting the physics. So even though the spline uses quadratic segments, it still produces a piecewise-linear-like behavior, just smoother and more refined.

In that context, yes—the quadratic spline is a decent and practical estimation. It doesn't violate the physics, and it provides a visually and numerically stable way to represent the temperature distribution across multiple segments.

Chapter 6: Regression

In this section, we apply quadratic regression to model the temperature distribution across the length of the composite wall. Unlike interpolation, which forces the curve to pass through all known data points, regression focuses on fitting a general trend through the data by minimizing the overall error. This approach is particularly useful when the data may contain slight measurement inconsistencies or when a smooth trendline is more desirable than exact interpolation.

```

N = length(L_plot);
sum_x = 0;
sum_x2 = 0;
sum_x3 = 0;
sum_x4 = 0;
sum_y = 0;
sum_xy = 0;
sum_x2y = 0;
for i = 1:N
    sum_x = sum_x + L_plot(i);
    sum_x2 = sum_x2 + L_plot(i)^2;
    sum_x3 = sum_x3 + L_plot(i)^3;
    sum_x4 = sum_x4 + L_plot(i)^4;
    sum_y = sum_y + T(i);
    sum_xy = sum_xy + T(i)*L_plot(i);
    sum_x2y = sum_x2y + T(i)*(L_plot(i)^2);
end
P = [N sum_x sum_x2; sum_x sum_x2 sum_x3; sum_x2 sum_x3 sum_x4];
R2 = [sum_y; sum_xy; sum_x2y];
Q = P\R2;
x_fit = linspace(min(L_plot), 0.08, 1000);
y_fit = Q(3) * x_fit.^2 + Q(2) * x_fit + Q(1);
figure(5)
hold on;
plot(x_fit, y_fit, 'k', 'LineWidth', 1.5);
xlabel('Length'); ylabel('Temperature'); title('Quadratic Regression');
set(gca, 'FontSize', 14);
legend('Interpolated Points', 'Spline Fit', 'Regressed Line')
grid on;
for i = 1:N
    residual(i) = T(i) - (Q(3) * L_plot(i)^2 + Q(2) * L_plot(i) + Q(1));
end
figure(6)
plot(L_plot, residual, 'k', LineWidth=1.5);
xlabel('Lengths'); ylabel('Residuals');
title('Residuals of Polynomial Fit');
grid on;
set(gca, 'FontSize', 14);

```

Figure 15: Code to apply a quadratic regression model

process reinforces the underlying mathematical concepts behind regression and avoids black-box solutions. Once verified, the results were compared against MATLAB's *polyfit* function for confirmation.

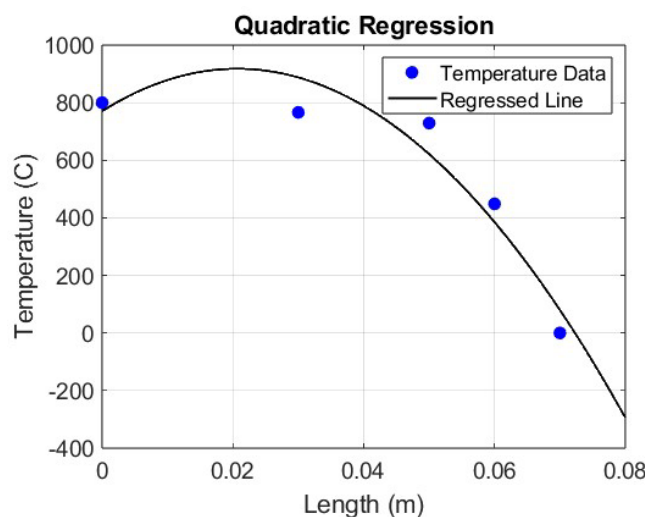


Figure 16: Regressed Line with Temperature Data Points

residuals were plotted as a function of position to assess how well the model fits across the full domain.

A quadratic regression model assumes that the best fit line between temperature and distance data can be described by a second-degree polynomial of the form:

$$f(x) = ax^2 + bx + c$$

The goal is to determine the coefficients a , b , and c such that the total residual—the vertical distance between each data point and the fitted curve—is minimized. Specifically, we minimize the sum of squared residuals, which ensures that large errors are penalized more heavily than small ones. This is a standard technique in least squares regression and provides the best-fitting curve in the sense of minimizing the total squared error.

To find the coefficients manually in MATLAB as shown in **Figure 15**, the system of normal equations was constructed and solved directly without using the built-in *polyfit* function. This

The fitted quadratic curve was then plotted alongside the original temperature data points can be seen in **Figure 16**. Unlike the interpolated splines that pass exactly through each point, the regression curve provides a smooth approximation of the overall temperature trend across the material.

Additionally, the residuals—differences between the actual data points and the predicted values from the regression model—were calculated at each data point this code can also be referred to in **Figure 15**. These

Figure 17 shows the residuals from the quadratic regression model plotted as a function of position along the composite wall. Residuals represent the difference between the actual temperature values and the values predicted by the regression model. From the graph, we can observe that the residuals vary both above and below zero, indicating that the model both underestimates and overestimates temperatures at different points. The alternating pattern of positive and negative values is typical for polynomial fits and suggests that while the model captures the general trend of the data, it doesn't perfectly match all points. The magnitude of the residuals also provides insight into the accuracy of the fit—larger deviations highlight where the model is less precise. This plot is a useful tool for assessing the goodness-of-fit and identifying regions where the model could potentially be improved.

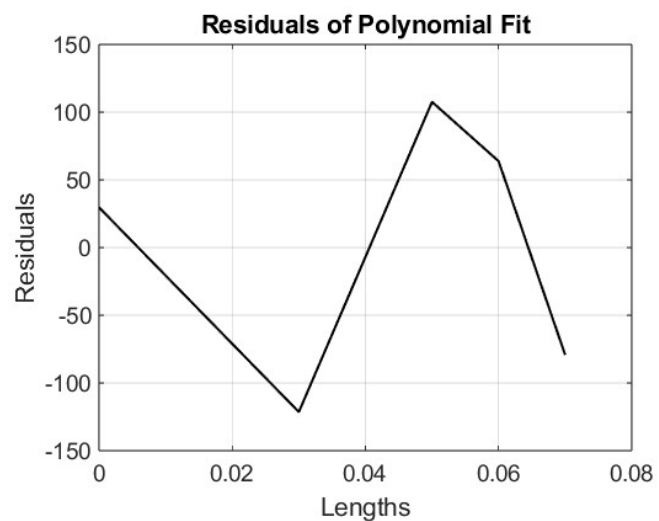


Figure 17: Residuals Vs Length

Question for Chapter 6:

1. Yes, the quadratic model as seen in **Figure 16** appears to be a reasonably good fit for the temperature data, though it is not perfect. From the residual plot **Figure 17**, we can observe that the residuals are relatively small near the middle of the domain (especially around 0.03 to 0.05 m), which suggests the model captures the overall trend well in that region. However, the residuals become larger towards the edges, especially near 0.02 m and 0.06 m, where the errors reach values of over 100. This indicates that the quadratic model fails to perfectly capture the curvature of the data at the ends.

Overall, because the residuals don't deviate wildly or show a strong pattern of systematic error (like a curve that the model completely misses), the quadratic model can be accepted as a decent approximation, particularly for a second-order polynomial regression.

2. The residuals shown in **Figure 17** exhibit a clear oscillatory behavior—alternating above and below zero. This oscillation is expected and reflects a core principle of least squares regression: the goal is not to pass exactly through every data point, but to minimize the sum of squared residuals. This means some points will fall above the fitted curve (positive residuals) and some below (negative residuals). The alternating pattern suggests that the regression is balancing the errors across the dataset, rather than accumulating bias in one direction.

Part Three: Introduction to Variable Thermal Conductivity Integration

In this section, we explore how heat transfer is affected when a material's thermal conductivity is not constant but instead varies with position. This type of situation commonly arises in engineering applications involving composite or non-homogeneous materials. To estimate the total heat transferred through such a material, we rely on numerical integration techniques. Specifically, we use the trapezoidal rule to approximate the total heat conducted across the wall. The focus of this part is to accurately model the spatial variation in conductivity and determine the minimum number of subdivisions required to meet a specified error threshold, providing both practical insights and computational experience.

Chapter 7: Numerical Integration Using Trapezoidal Rule

In this final portion of the chapter, we examine a wall composed of a material whose thermal conductivity varies as a function of position, given by:

$$k(x) = x^{-x} \cdot \ln(x + 1.2)$$

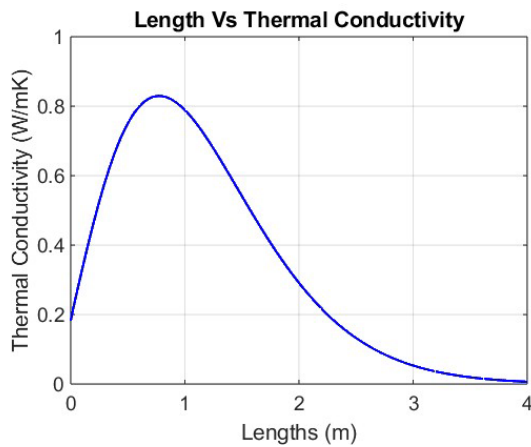


Figure 18: Thermal Conductivity Vs Length

To begin the analysis, the thermal conductivity function $k(x)$ was plotted over the domain $x = 0$ to $x = 4$ meters, as shown in **Figure 18**. The curve illustrates how thermal conductivity varies along the wall, initially rising to a peak near $x = 1$ before steadily declining toward zero. This non-linear behavior emphasizes the need for numerical integration, as the changing slope and curvature would make analytical solutions impractical. The plot provides a clear visual reference for the spatial variation in material properties that directly influences the total heat

transfer.

To determine the total heat transfer Q through the wall over the domain $x = 0$ to $x = 4$ meters, we must evaluate the integral:

$$Q = \int_0^L k(x) dx$$

Since the function is non-polynomial and not easily integrable by hand, we utilize numerical integration, specifically the trapezoidal rule, to approximate the integral.

The trapezoidal rule works by approximating the area under a curve as a series of trapezoids rather than rectangles. The area of each trapezoid is computed based on the average height of the function at the endpoints and the width of the interval. Summing these areas provides an estimate of the total integral. While this method is efficient, its accuracy depends on the number of trapezoids used. Increasing the number of subdivisions N decreases the error in the approximation.

```
Lnew = 4;
x = 0:0.001:Lnew;
k = [];
for i = 1:length(x)
    k(1,i) = (x(i)^(-x(i)))*log(x(i)+1.2);
end
plot(x, k, 'b', LineWidth=2);
xlabel('Lengths (m)');
ylabel('Thermal Conductivity (W/mK)');
set(gca, 'FontSize', 14);
title('Length Vs Thermal Conductivity');
grid on;
syms x1
expr = (x1^(-x1))*log(x1+1.2);
Q = double(int(expr, x1, 0, Lnew));
Qnew = @(x)((x.^(-x)).*log(x+1.2));
x_n = [];
N = 3;
abs_rel_true_error = 1000;
x_vals = [];
y_vals = [];
while abs_rel_true_error >= 0.00001
    dx = Lnew / N;
    x_vals = linspace(0, Lnew, N+1);
    y_vals = Qnew(x_vals);
    Trapez_sum = .5 * dx * (y_vals(1) + 2 * sum(y_vals(2:end-1)) + y_vals(end));
    abs_rel_true_error = abs((Q - Trapez_sum) / Q);
    if abs_rel_true_error > 0.00001
        N = N+1;
    end
end
fprintf('Number of trapezoids required: %d\n', N);
```

Figure 19: MATLAB code for Calculating Thermal Conductivity and Trapezoidal Rule

To ensure a highly accurate result, a MATLAB script was written as shown in **Figure 19** that iteratively increases the number of trapezoids starting from $N = 3$, recalculating the integral each time and comparing it to the true value obtained symbolically using MATLAB's *int* function. The loop continues until the absolute relative true error falls below 0.001%, guaranteeing a near-exact estimate of heat transfer.

This approach allowed us to automatically determine the minimum number of subdivisions required for the trapezoidal method to meet the desired accuracy threshold. The result was then compared against the true value to validate convergence. This process demonstrates how numerical methods—when implemented with adaptive iteration—can provide both precision and efficiency in solving real-world engineering problems where analytical solutions are impractical.

Table 5: Trapezoidal Rule Output

No. Trapezoids Required	451
-------------------------	-----

Conclusion:

This project showcased the importance and versatility of computational methods in solving complex engineering problems that are otherwise analytically intractable. From decoding fixed-point binary velocity data to evaluating the accuracy of experimental measurements against theoretical predictions, we developed a strong foundation in numerical representation and error analysis. The transition to thermal systems allowed us to apply matrix-solving techniques to model temperature profiles, and interpolation strategies such as polynomial fits and splines revealed the nuances of numerical stability and physical realism.

In regression, we saw how statistical techniques can model overall system trends even in the presence of noise or incomplete data, while standard error analysis provided a concrete metric for evaluating model adequacy. Finally, the use of the trapezoidal rule for integrating a position-dependent thermal conductivity function highlighted the practical relevance of adaptive numerical integration in real-world scenarios.

Altogether, this project emphasized not only the computational rigor but also the physical interpretability of numerical methods in fluid and thermal systems—reinforcing their value as indispensable tools in the engineering toolkit.